ZUZU and Claude Code: The One-Person, Two-Country Software Company

Hyun Jae Moon
Software Engineer
hyunjae@moonai.co.kr

July 15, 2026

Abstract

This paper is an experience report grounded in two phases of the author's work: three years of software engineering at Google (2022–2025), and the solo build of Moonai, a founder-led AI delivery company, developed through 2025 and officially launched in 2026, as a single person operating across two jurisdictions. Two observations anchor the report. First, a solo founder cannot realistically provide full application development (the sustained frontend, mobile, and product-design iteration a consumer-grade app demands) for every product in a portfolio; that capability gap is real and did not go away. Second, and more surprisingly, two pieces of infrastructure proved *sufficient* to deliver production backend systems software on both Google Cloud Platform (GCP) and Amazon Web Services (AWS) anyway: an agentic coding tool (Claude Code) that collapsed the engineering cost of backend delivery, and a corporate-operations platform (ZUZU, zuzu.network) that collapsed the administrative cost of running a Korean corporate entity. A third enabling condition, the founder's dual footing as a South Korean citizen and a United States permanent resident (green card holder), made it legally and practically possible to operate businesses in both countries simultaneously. We describe how these three elements compose, what they made possible, what they conspicuously did not solve, and what this implies for the minimum viable shape of a software company in the agentic-AI era.

1 Introduction

The conventional wisdom of the 2010s was that a software company is a team sport: a backend engineer, a frontend engineer, a mobile developer, a designer, an operations person, and someone to handle the legal and administrative machinery of incorporation, filings, and compliance. A founder missing any of those functions was expected to hire for it or fail at it.

Since 2022 I have run an experiment, mostly involuntarily, on how much of that structure is actually load-bearing: three years of software engineering at Google through 2025, much of it release engineering and CI/CD for network-simulation software, then the solo build of Moonai from 2025 onward. Moonai (branded in Korean as *Moon-AI*, with *Moon* written in Hangul) is an AI delivery company targeting Korean small and medium-sized businesses, and it has exactly one person: me. 2025 was the year of development; 2026 is the official launch, when the full process (US LLC formation, the Korean *beopin*, supplier registration, and the first paying engagements) comes together. Building it, I have attempted to stand up a portfolio of products, drawing on the full-time engineering perspective those Google years produced: an AI-powered industrial catalog (yushin-melamine.com), a social game-tracking platform (playshelfapp.com), an autonomous Python testing agent (pythontestingagent.com), consumer AI experiences, and the delivery business itself.

The honest result is asymmetric. I was *not* able to provide app development at full product scale for the various Moonai products: the polish loop of consumer application development, with its design iteration, platform-specific mobile work, and daily grind of UI refinement that separates a demo from a product people live in, exceeds what one person can sustain across a portfolio. But I *was* able to provide production-grade backend systems software on both GCP and AWS, deployed, monitored, multi-region, and serving real customers. The difference between those two outcomes was not talent or effort. It was that two specific pieces of infrastructure existed for one side of the work and not the other.

This paper names those two pieces (Claude Code for engineering delivery and ZUZU for corporate operations), examines why they were sufficient, situates them within the cross-border structure that

my dual status as a South Korean citizen and a US green card holder made possible, and generalizes the lesson: the minimum viable software company is now defined by which of its functions have been absorbed into sufficient infrastructure, and which have not.

2 Background: One Founder, Two Countries, One Portfolio

2.1 The cross-border position

I am a citizen of the Republic of Korea and a lawful permanent resident (green card holder) of the United States. This combination, which for most of my career felt like an administrative burden (two tax regimes, two sets of filings, residency obligations on the US side), turned out to be the structural precondition for everything that follows.

Korean citizenship makes me a first-class participant in the Korean business ecosystem: I can establish a Korean corporation (a *beopin*), register as a supplier for government-subsidized digitalization programs, and sell to Korean SMBs inside a procurement and subsidy system that is effectively closed to foreign entities. US permanent residency makes me a first-class participant in the American one: I can reside in California, form a single-member LLC as a day-one invoicing vehicle, open US banking and payment rails, and contract with US customers without visa risk.

Neither country's position alone would support Moonai's model. The market and the delivery playbook are Korean; the founder's residence, early cash flow, and cloud billing relationships are American. Being legally native to both sides is what lets a one-person company behave, jurisdictionally, like a company with an international subsidiary structure.

2.2 The portfolio and the capability gap

Moonai's proof-of-capability portfolio spans several production systems. The flagship, an AI-powered bilingual product catalog for a Korean melamine manufacturer, runs on GCP: Cloud Run services in `us-central1` and `asia-northeast3`, Vertex AI Gemini for search and chat, Firestore for catalog data, Google Cloud Storage for assets, BigQuery for file metadata. The company's own web presence runs on AWS: CloudFront in front of S3 and a Lambda function URL, with managed Postgres for persistence, an architecture chosen to sit inside free tiers. The delivery playbook itself is explicitly dual-cloud: the same module library maps onto Cloud Run + Vertex AI + Firestore + GCS on GCP and onto App Runner/Fargate + Bedrock + DynamoDB + S3 on AWS.

What the portfolio does *not* contain is what I could not staff: sustained application development. Playshelf, the social game-tracking product, has a real backend (FastAPI, PostgreSQL with migrations, rate-limit-aware background workers), but its consumer-facing application surface advanced at a fraction of the pace a funded consumer startup would demand, and it remains invite-only. Several product concepts never got frontends at all. The pattern repeated enough times to be a law rather than an accident: **as a solo founder, backend systems software was deliverable; app development at product scale was not.**

The interesting question is why the line fell exactly there.

3 Claude Code: Agentic Coding as the Missing Backend Team

3.1 What changed

The first answer is Claude Code, the agentic coding tool I adopted as it matured over this period. The distinction between an AI code assistant and an agentic coding tool matters here. An assistant accelerates the person typing; an agent executes multi-step engineering work under human direction rather than human keystrokes: reading a codebase, writing code across many files, running commands, reading the errors, and iterating.

Backend systems work turned out to be the domain where this is most transformative, for reasons intrinsic to the work:

- **Backend correctness is machine-checkable.** A Cloud Run service either serves requests or does not; an IAM policy either permits the invocation or returns 403; a migration either applies or fails. The feedback an agent needs in order to iterate autonomously is produced by the system

itself, immediately and unambiguously. Deployment scripts, infrastructure inventories, and API contracts close the loop.

- **Cloud platforms are text all the way down.** GCP and AWS are operated through CLIs, declarative configuration, and well-documented SDKs, which is exactly the medium a language model is strongest in. The two providers' services are also near-isomorphic (Cloud Run ↔ App Runner, Firestore ↔ DynamoDB, Vertex AI ↔ Bedrock, GCS ↔ S3), so an agent that has built a system on one can be directed to map it onto the other. Moonai's "one playbook, two clouds" principle is only economical for one person because the mapping work is agentic, not manual.
- **The founder's judgment remains the scarce input, and backend judgment compresses well.** My role shifted from writing code to specifying architecture, reviewing diffs, and owning the security and cost posture: secret management, auth patterns, region strategy, free-tier discipline. One experienced person's judgment, amplified by an agent, covers the backend surface of an entire small portfolio.

The concrete outcome: multi-region deployment templates, a retrieval-augmented search pipeline, JWT auth with secret-manager backing, image pipelines, ingestion crawlers, and serverless web infrastructure, on two clouds, were built, deployed, and operated by one person. A few years ago this was a three-to-five-engineer team's output. That is the precise sense in which Claude Code was *sufficient*: not that it wrote every line, but that the combination of one founder plus one agentic tool cleared the entire backend delivery bar that Moonai's customers required.

3.2 Why the same tool did not close the app development gap

It is tempting to expect the same leverage on the application side, and the five-year record says otherwise. App development at product scale failed to compress for reasons that are the mirror image of why backend work compressed:

- **The feedback loop runs through humans.** Whether a screen feels right, whether an onboarding flow loses people, whether a design language is coherent across forty views: these verdicts come from users and taste, not from exit codes. The agent can produce candidate UIs faster than ever, but the evaluation bottleneck is the founder's attention, which does not scale.
- **Product surface area is open-ended.** A backend has a finite contract; a consumer app is an unbounded commitment to polish, platform updates, store review cycles, and the long tail of device-specific behavior. Sufficiency for a backend is a passing deployment; sufficiency for an app is a retention curve.
- **Solo context-switching is the real cost.** Even with generation nearly free, one person alternating between operating production backends for paying customers and doing design-led product iteration pays a switching penalty that compounds. The backend work, being interrupt-driven and verifiable, survives fragmentation; creative product work does not.

The strategic response was to stop fighting the asymmetry. Moonai's business model is the shape of the tooling: it sells *backend-heavy AI systems* (audits, six-week MVP sprints on the reference stack, and release-engineering retainers) rather than open-ended app development. The products I could not fully build became proof-of-capability case studies rather than revenue lines. Knowing which half of the company the tools had made buildable, and building only that half, is itself the central piece of knowledge this build produced.

4 ZUZU: Corporate Operations as a Platform

4.1 The administrative wall

Engineering was only half the problem. A Korean-market business run by a US-resident founder faces a corporate-administrative burden that has historically required professional staff: establishing and maintaining a Korean corporate entity, keeping the shareholder ledger, executing corporate registration filings (*deunggi*) whenever the company's registered facts change, and producing the documentary trail that

Korean banks, government subsidy programs, and enterprise customers demand before they will transact. In the traditional model this means a judicial scrivener for every filing, an accountant for every ledger event, and ideally an administrative employee to coordinate them. That is a payroll a pre-revenue one-person company cannot carry, and a coordination load a founder twelve time zones away cannot absorb.

This wall is not incidental. Korea's subsidy ecosystem (the smart-factory and voucher programs that make AI projects affordable for Korean SMBs) runs through registered domestic suppliers. A supplier registration requires a real, compliant, current Korean corporation. No compliant entity, no market access. For Moonai, corporate administration was therefore not overhead; it was the gate to the entire go-to-market.

4.2 What ZUZU absorbed

ZUZU (zuzu.network) is a Korean corporate-operations platform that turns this administrative layer into software: incorporation workflows, shareholder-ledger management, corporate registration filings, stock-option and equity records, and the generation of the legal documents that each of these events requires. The experience of using it, from the founder's seat, is closely analogous to the experience of using a cloud provider: a function that used to require hiring is invoked on demand, priced per use, and executed correctly without the founder needing to become an expert in its internals.

Three properties made it sufficient for the cross-border case specifically:

- **Asynchronous and remote by default.** Filings and ledger events are prepared, reviewed, and executed through the platform rather than through in-person meetings in Seoul. For a founder resident in California, this is the difference between a viable Korean entity and an impossible one.
- **Correct-by-construction paperwork.** Korean corporate filings are unforgiving of format errors, and rejection cycles are measured in weeks. Encoding the requirements in software removes the class of failure that a non-specialist founder would otherwise generate constantly.
- **Legibility to the ecosystem.** Because the entity's records are maintained in a standard, current, professionally-legible form, downstream interactions (bank accounts, subsidy applications, supplier registrations, eventual investor diligence) start from a clean documentary state rather than a remediation project.

The parallel to Claude Code is exact and is the thesis of this paper: **ZUZU is to the corporate-operations function what Claude Code is to the backend-engineering function**, a platform that absorbs an entire role a company previously had to staff. On the US side the same pattern held with commodity components: a California single-member LLC, a commercial registered agent, and an EIN assembled a compliant invoicing vehicle in days for a few hundred dollars. Neither jurisdiction required me to hire anyone.

5 The Supporting Cast: Managed Models, Payments, and Local Inference

ZUZU and Claude Code carried the two functions, corporate operations and backend engineering, that would otherwise have required hiring. But the five-year record would be incomplete without naming the layer of infrastructure beneath them that was, plainly, amazing to build on. Three categories deserve explicit credit.

5.1 Managed model APIs: GCP Vertex AI and Amazon Bedrock

Every AI feature Moonai shipped runs on a managed model API: Vertex AI (Gemini) on the GCP side and Amazon Bedrock on the AWS side. These services are the reason a one-person company can sell "AI systems" at all. There is no model to train, no GPU fleet to provision, no serving stack to keep alive at 3 a.m.; the entire machine-learning-infrastructure function arrives as an endpoint with per-token pricing. Vertex AI in particular proved outstanding in production: the Gemini model family powers the flagship catalog's semantic search, chat, and description generation, with model tiers (`flash` for chat, `flash-lite` for search scoring) that let cost be tuned per feature. Bedrock provides the same shape on AWS, which is exactly what makes the "one playbook, two clouds" principle real: the playbook

can promise a customer either cloud because both clouds offer a first-class managed model service with near-identical integration ergonomics. Just as importantly, per-token pricing is what pulled AI projects inside Korean SMB budgets in the first place. The economic premise of Moonai's market exists because these two services exist.

5.2 Payments: Stripe

On the commercial side, Stripe did for the payments function what ZUZU did for the corporate one. With nothing more than the California LLC and its EIN, Stripe provided invoicing, card and bank-transfer acceptance, and cross-border billing: the entire accounts-receivable machinery that a company traditionally grows a finance function to operate. For a founder invoicing from a US entity while selling into relationships that span two countries, having payments arrive as an API with the compliance, tax-form, and payout plumbing handled underneath was not a convenience but a precondition: it is the reason the day-one LLC could actually collect revenue on day one. Stripe belongs on the same list as the cloud providers: infrastructure good enough that an entire business function never had to be staffed.

5.3 Local inference: Ollama and MLX-optimized models

Not every model call belongs in the cloud. Ollama, running local model variants including MLX-optimized builds that exploit Apple Silicon's unified memory, turned the founder's own machine into a zero-marginal-cost inference environment, and it was fantastic for exactly the work that per-token pricing discourages: rapid prototyping of prompts and pipelines before committing them to Vertex AI or Bedrock, offline development while traveling between the two countries, and demos for privacy-sensitive Korean SMB prospects whose first question is often whether their data must leave the building. The pattern mirrors the rest of the paper at a smaller scale: local inference did not replace the managed services (production traffic runs on Vertex AI and Bedrock), but it absorbed the experimentation function, making the expensive layer something you arrive at deliberately rather than iterate against.

6 Composition: Why the Three Elements Only Work Together

Each element in isolation is insufficient, which is why the five-year experience felt like a discovery rather than an obvious plan.

- **Claude Code without ZUZU** produces deployed systems with no compliant entity to sell them through; in the Korean market especially, running software without a running corporation is commercially inert.
- **ZUZU without Claude Code** produces a perfectly administered company with no product, since the founder alone could not have delivered the dual-cloud backend portfolio by hand at acceptable speed.
- **Both tools without the dual citizenship-residency footing** strand the operation on one side of the Pacific: without Korean citizenship, the Korean entity, subsidy access, and SMB trust-building become dramatically harder; without US permanent residency, the founder cannot lawfully reside where he does, anchor early revenue in a US LLC, or maintain the American professional network the delivery skills came from.

Composed, they form a company whose org chart is: one human (judgment, sales, architecture, taste), one agentic engineering platform (backend delivery on GCP and AWS), one corporate-operations platform per jurisdiction (ZUZU in Korea; LLC + registered-agent stack in the US), and two passports' worth of legal standing (more precisely, one passport and one green card). That structure delivered production systems for real customers on two clouds while remaining, in headcount terms, a single person.

7 Lessons

1. **Sufficiency is the right bar, not capability.** The question that mattered was never "can AI code?" or "can software file paperwork?" but "is the combination sufficient to clear the customer's

bar with zero additional hires?" For backend systems software plus Korean corporate operations, by 2026, the answer became yes. That is a statement about thresholds, and thresholds are what change business models.

2. **Know which functions have been absorbed into infrastructure, and shape the company around the boundary.** Backend engineering and corporate administration crossed the line; app development, design, and sales did not. Moonai's product catalog (audits, backend-heavy MVP sprints, release-engineering retainers) is drawn exactly along that boundary. A solo founder who shapes the offering around what the infrastructure absorbs can be credible; one who pretends the boundary is not there ships half-finished apps.

3. **Verifiability predicts which work compresses.** The deep reason backend work compressed and app work did not is that agents thrive where the environment itself judges the output. When choosing what a small company should sell in the agentic era, "how mechanically checkable is success?" is a better guide than "how hard is the work?"

4. **Dual-jurisdiction standing is an asset, not overhead.** For years the citizen-plus-green-card position looked like double taxation paperwork. In practice it was the moat: very few people can simultaneously be a domestic Korean supplier and a US-resident operator, and the position cannot be acquired quickly by a competitor.

5. **The founder's remaining job is judgment, and it must be guarded.** Every function the platforms absorbed returned time that only mattered if it flowed into the functions they cannot absorb: architecture decisions, security posture, pricing, customer trust, and knowing what *not* to build. The one-person company does not eliminate the founder's work; it purifies it.

8 Limitations and Open Questions

This is a single-founder, single-company experience report; it generalizes with care. The founder entered Moonai with three years of software engineering at Google (2022–2025); the agentic leverage described here multiplies existing judgment and may not bootstrap it. The app-development gap is described as of 2026; agentic tooling for design-led product work is improving, and the deliverability boundary drawn in the lessons above should be treated as a moving frontier rather than a law of nature. The corporate-operations claim is specific to the Korean and Californian regimes actually exercised; other jurisdictions may lack a ZUZU-equivalent, and the dual-status advantage is by definition not replicable on demand. Finally, both sufficiency claims inherit platform risk: a one-person company built on two platforms has concentrated its dependencies as much as it has reduced its payroll, and pricing or policy changes at either layer propagate directly into the company's cost base.

9 Conclusion

Building Moonai alone across South Korea and the United States, developed through 2025 and launched in 2026 on a foundation of three years of software engineering at Google, taught me one compact fact: a solo founder in 2026 can deliver production backend systems software on GCP and AWS, and operate compliant corporate entities in two countries, because two categories of platform (agentic coding tools exemplified by Claude Code, and corporate-operations platforms exemplified by ZUZU) have absorbed functions that used to be headcount. Beneath them, an equally remarkable substrate absorbed the machine-learning, finance, and prototyping functions the same way: Vertex AI and Bedrock for managed models, Stripe for payments, and Ollama with MLX-optimized local models for experimentation. The same experience taught me the boundary of that fact: full application development for a product portfolio remained beyond one person, and the durable move was to design the business along the boundary rather than across it. The one-person, two-country software company is no longer a stunt. It is what happens when a founder's legal standing spans the markets, the engineering is verifiable enough for agents, and the paperwork has become an API.